



# From Cron to Assets:

## Event-Driven Drone Telemetry Ingestion with Airflow



Charles Adetiloye and Bryan Johns | MavenCode

# Who is talking



## Charles Adetiloye

**Principal AI Engineer, MavenCode**

As Co-Founder and senior technical lead at MavenCode, Charles holds an MSc in Computer Systems Engineering from Northeastern University. With 20+ years building large-scale ML and distributed data systems, he's consulted for Google, AWS, X, Fortune 500 firms, and the US federal government—delivering autonomy software, sensor fusion, and edge AI solutions. He speaks widely on Agentic AI and distributed ML infrastructure.

## Bryan Johns

**AI Product Manager, MavenCode**

Bryan leads cross-functional AI and engineering teams across complex technical programs, driving delivery from requirements through deployment. A proven operations leader, he aligns engineers and executives around shared outcomes, translating technical progress into clear direction and decision-ready updates while managing schedule, risk, and program execution to a successful finish.

# A drone doesn't care what time it is.

It takes off when the mission says so, lands  
when the battery says so,  
and uploads its logs whenever the LTE link or  
WiFi finally cooperates.



# Commercial drone fleet operations



## Infrastructure inspection

Powerline, pipeline, and tower corridors flown on repeatable routes. Anomalies must surface the same day the sortie flies.



## Surveying and mapping

Photogrammetry and corridor-mapping sorties. Deliverable quality depends on verified flight health for every pass.



## Incident response

On-demand launches with no schedule at all. Telemetry arrives whenever the mission ends and the link allows.

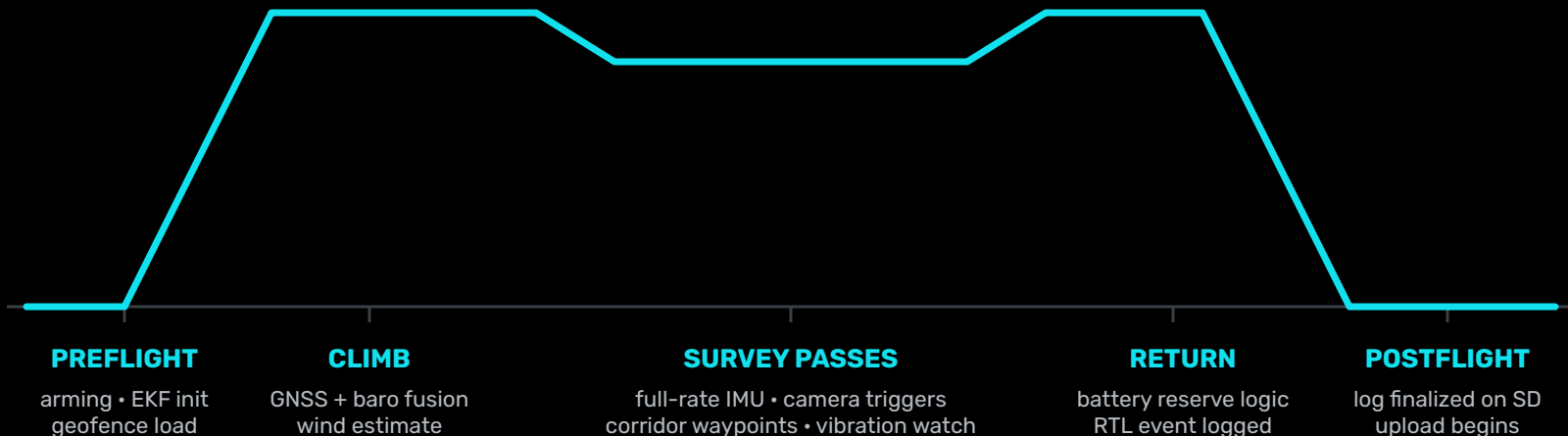
**Different missions, one identical data problem: every sortie ends with a full-rate flight log that must be ingested, validated, and evaluated.**

**Reference deployment used throughout:** inspection fleet • dozens of sorties per day across two mission blocks • tens to hundreds of MB per flight • uploads land 2 to 35 min after wheels-down

# Anatomy of a sortie



**every phase writes telemetry**  
one continuous ULog, armed to disarmed



**The log is one unbroken record of the whole sortie.** There is no meaningful partial flight to process.

# Four properties that fight a scheduler



## Bursty

An hour of silence, then two dozen flights landing in twenty minutes. Any fixed schedule is too slow for the burst and wasteful in the silence.



## Edge-originated

The data is born on an aircraft. The platform's first sight of it depends on a radio link the platform does not control.



## Session-shaped

The unit of meaning is a complete flight. A half-uploaded log parses cleanly and produces a confident, wrong summary.



## Long-tailed

Most uploads finish in under two minutes. The tail runs past thirty. Hold a resource per upload and you pay for the tail on every flight.

# From airframe to cloud: the delivery path



## Flight Controller

PX4 / ArduPilot

The complete ULog leaves the airframe only after landing. A 20-minute sortie is tens to hundreds of MB.



## Companion Computer

Onboard Processing

Buffers, compresses, and uploads the data. Writes the UPLOAD\_COMPLETE marker as its final act.



## Object Storage

raw/{flight\_id}/ + marker

The link is whatever the site offers: LTE, station WiFi, or an SD card pulled hours later.

[ Pipeline Ingestion Point ]



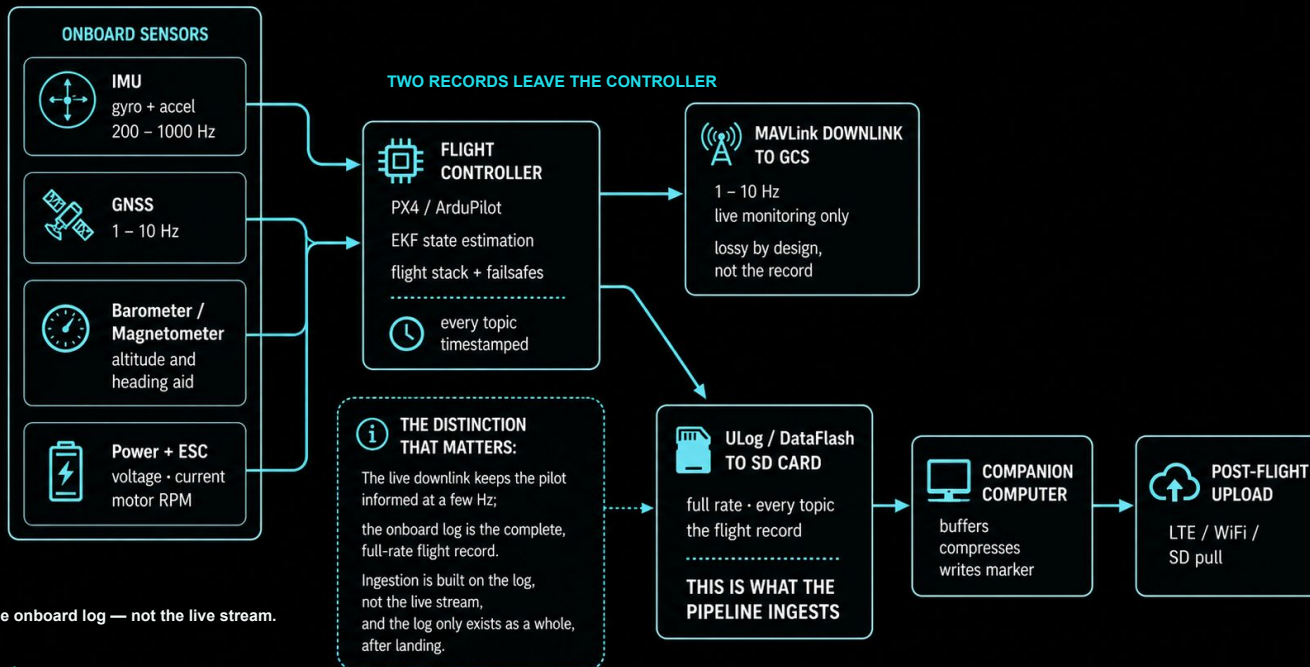
## Airflow 3 Platform

Orchestrated Downstream

Automatically orchestrates log parsing, flight health evaluation, perception-model retraining, and Part 107 reporting.

# How drone telemetry happens

## ONBOARD SENSORS



The pipeline ingests the onboard log — not the live stream.

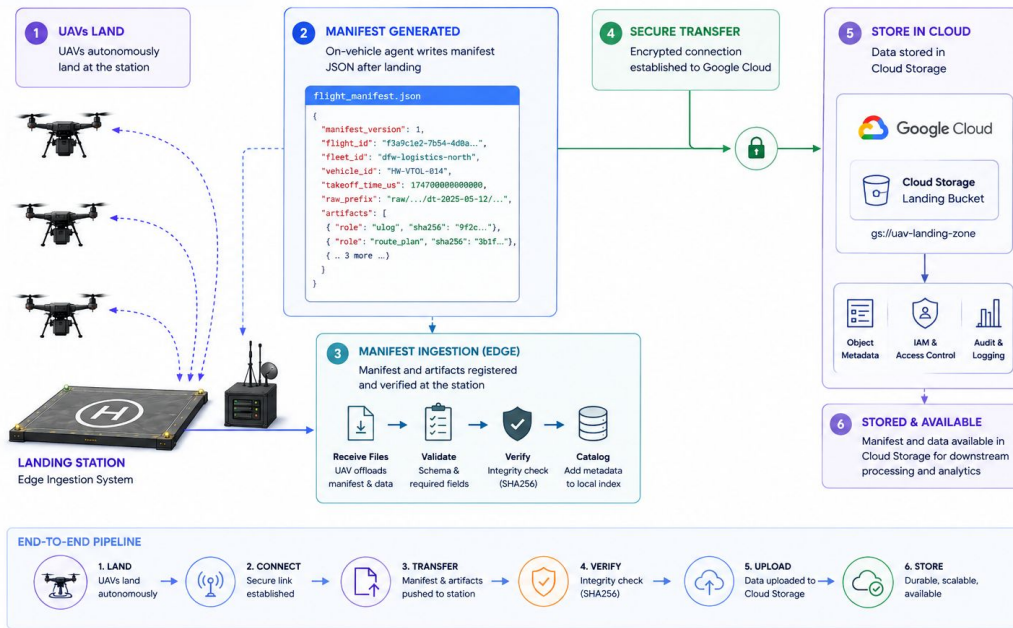
Complete log = source of truth.



# From airframe to cloud: the delivery path

## UAV MANIFEST INGESTION PIPELINE

From UAV Landing to Cloud Storage



# The five-minute DAG



```
@dag(schedule="*/5 * * * *", catchup=False)

def ingest_flights():
    new_logs = list_raw_prefix() # anything new?
    for log in new_logs:
        parse(log) # hope it's complete
```

**01** Poll object storage every five minutes.

**02** Process anything not seen before.

**03** This is what almost everyone builds first.

**It works fine.**  
**Until the fleet grows.**

# Empty polls

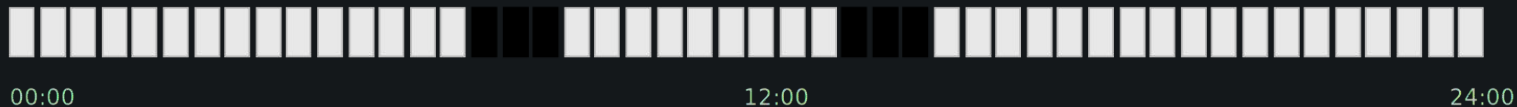


# 288

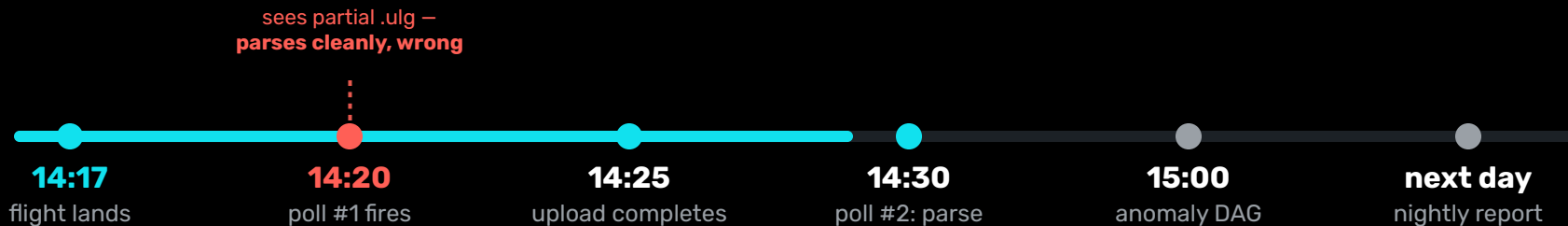
scheduler runs per day, against a fleet that flies in bursts

- The large majority of runs find nothing at all.
- Each still pays scheduler overhead, task startup, a LIST call, and a log line.
- The DAG is almost always green and idle, training on-call engineers to stop looking.

one day of five-minute polls: 288 wake-ups



# A flight through the polling world



**13 min** from landing to a full parse

**poll-bound**

*waiting on the poll*

**43 min** to the first finding

**schedule-bound**

*compounds per hop • waiting on downstream schedules*

*"Every gap is a clock: first the poll interval, then each downstream DAG's own schedule."*

# Throw the schedule away.



## Asset-Driven Workflows

In Airflow 3, every completed flight becomes a first-class **Asset**.

Downstream DAGs wake **only when the flight they care about** actually exists.

# Airflow 3 is built for events



## Assets, first-class

Datasets grew up into Assets: URIs, events with metadata, aliases, and an @asset authoring style. The data object is now the scheduling primitive.



## Event-driven Scheduling

Schedule DAGs on Assets and compose conditions with & and |. AssetWatcher brings external events straight into the scheduler.



## Waiting, solved properly

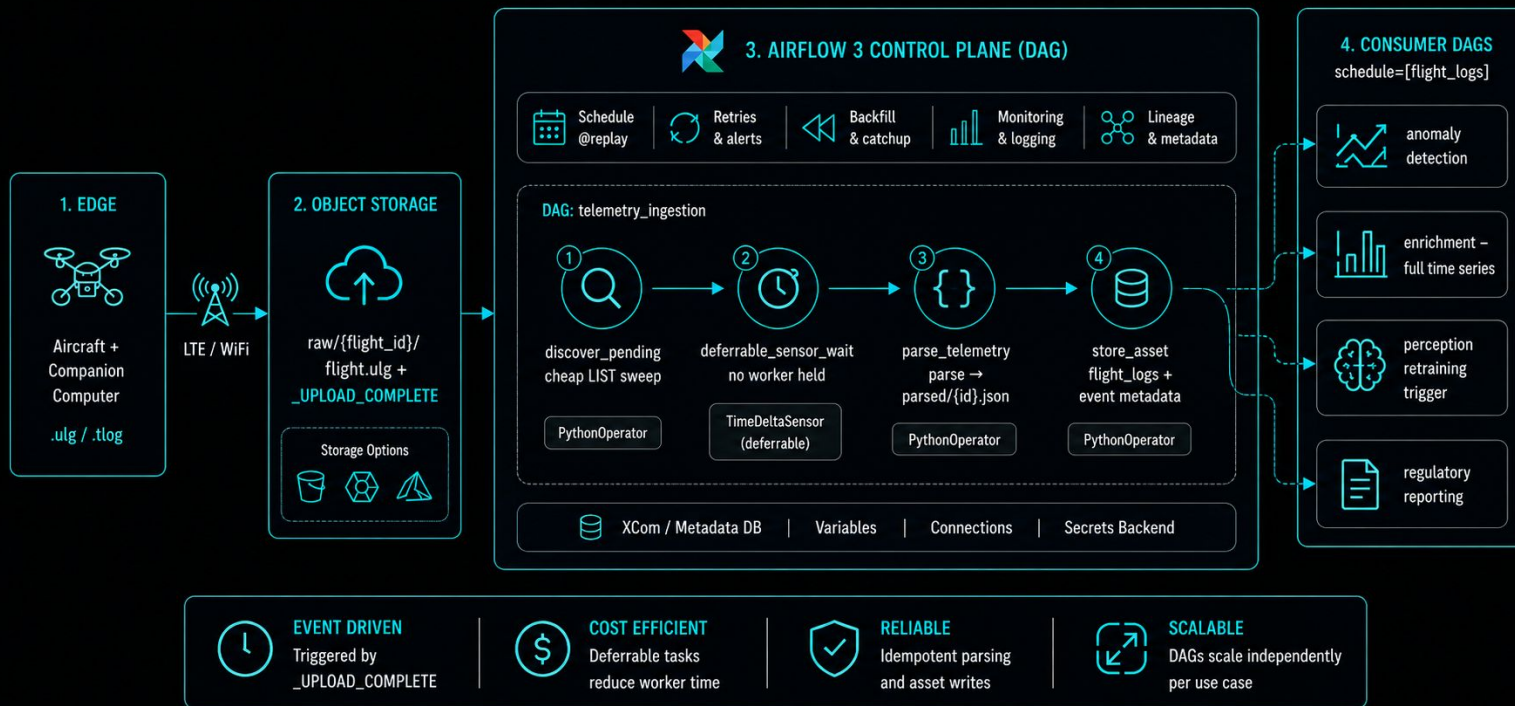
The deferrable Triggerer model matured into the default answer for every wait, with thousands of them on one async loop.



## A re-architected platform

Task SDK, DAG versioning, and a new UI. The release restructured Airflow around events rather than adding a feature.

# Target Architecture



# Overview of the Flight Log Assets



## Asset

a named data object, identified by a URI

## Outlets

how a task declares the Assets it produces

## Schedule

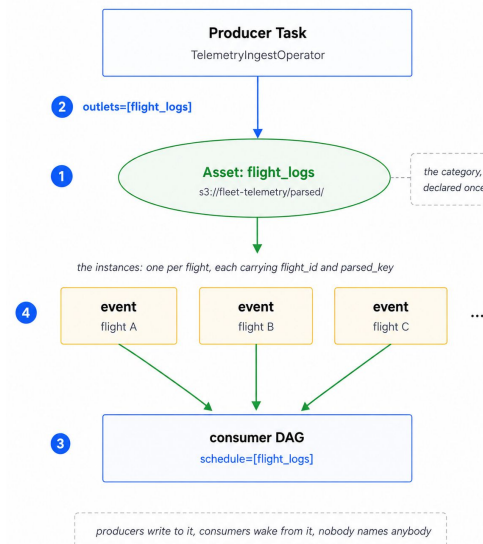
how a DAG declares the Assets it consumes

## Event

recorded on producer success; carries metadata the consumer reads

```
flight_logs = Asset("s3://fleet-telemetry/parsed/")
```

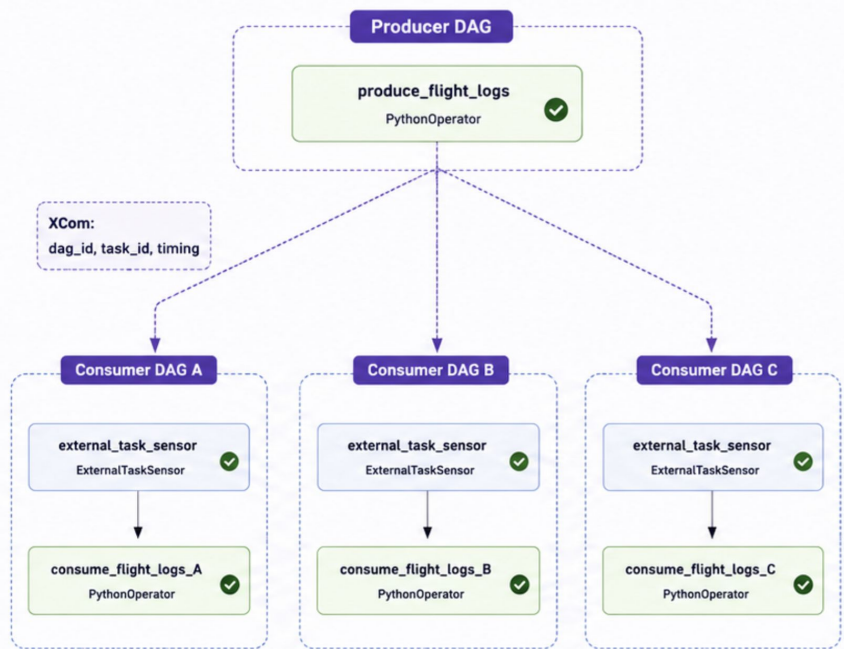
**One Asset for the whole namespace. One event per flight. The Asset is the category; the event is the instance.**



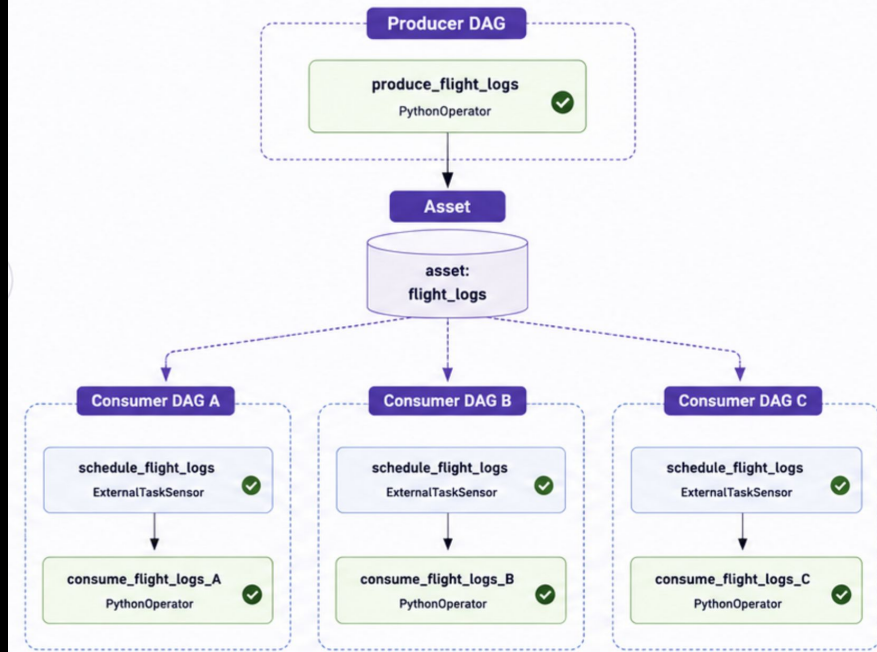


# What changed with Airflow 3 Assets

## BEFORE – Every consumer couples to the producer



## AFTER – One asset in the middle



# Event metadata



## PRODUCER ATTACHES

```
yield Metadata(  
    flight_logs,  
    extra={  
        "parsed_key": self.parsed_key,  
        "flight_id": parsed["flight_id"]  
    }  
)
```

## CONSUMER READS

```
events = context[  
    "triggering_asset_events"  
][flight_logs]  
  
parsed_key = events[-1].extra[  
    "parsed_key"  
]
```

## EVENT SCHEMA

```
each event → {  
    "flight_id": "<id>",  
    "parsed_key": "parsed/<flight_id>.json"  
}
```

If the storage layout ever changes, **only the producer changes**. Consumers never knew the layout at all.

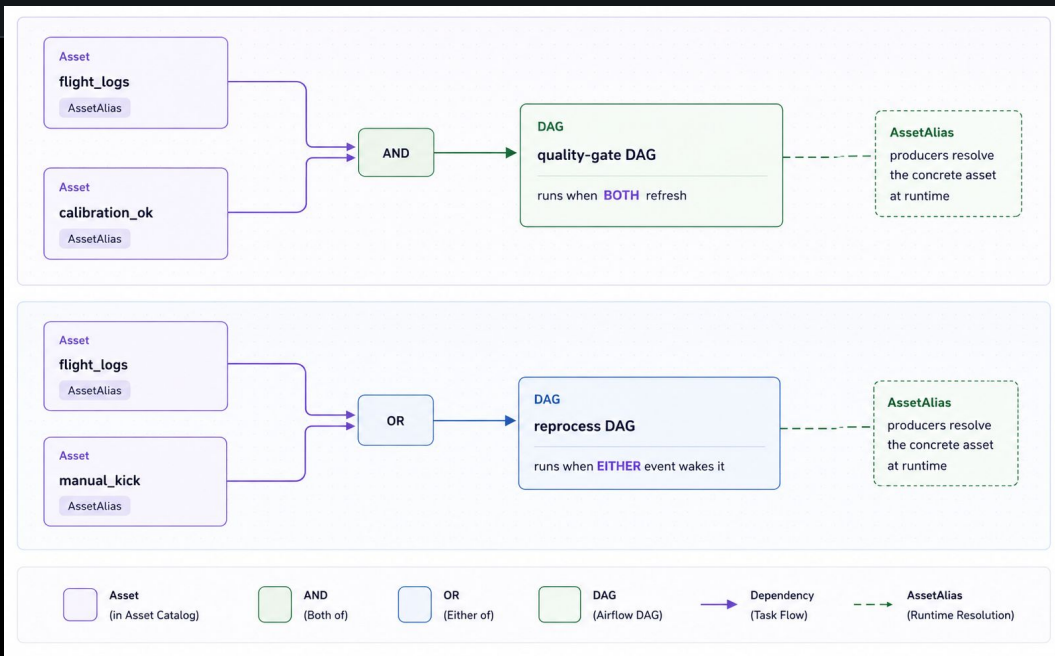
# Composing schedules on Assets



```
from airflow.sdk import Asset, AssetAlias
```

```
@dag(schedule=(flight_logs & calibration_ok))    # BOTH must refresh
```

```
@dag(schedule=(flight_logs | manual_kick))        # EITHER wakes it
```



# Watchers: when the edge can push



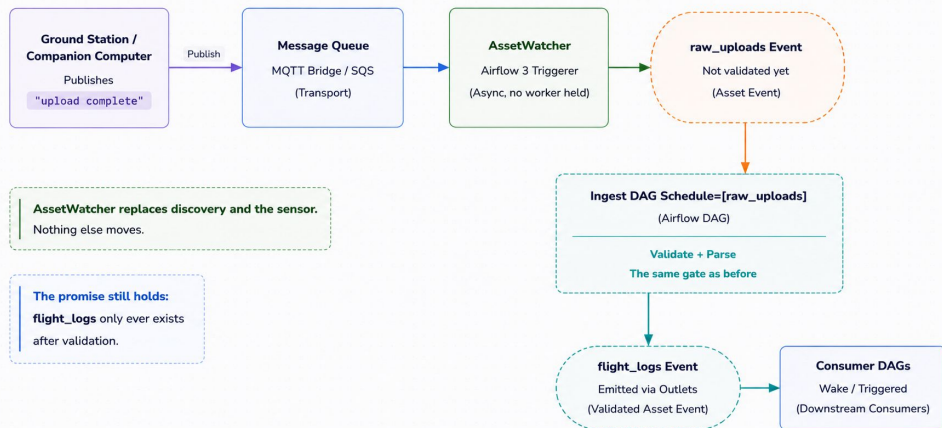
```

# Define the trigger on the message queue
trigger = MessageQueueTrigger(
    queue="fleet-uploads"
)

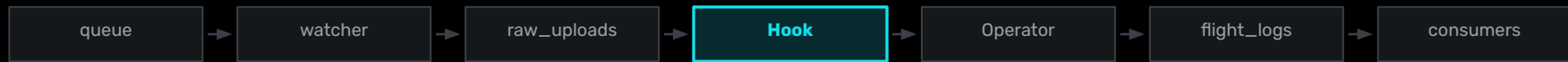
# Watch S3 and notify on trigger
raw_uploads = Asset(
    "gs://fleet-telemetry/raw/",
    watchers=[
        AssetWatcher(
            name="uploads",
            trigger=trigger
        )
    ]
)

@dag(schedule=[raw_uploads])
# wakes on the queue message
def ingest(): ...
```

## PUSH-BASED FLOW



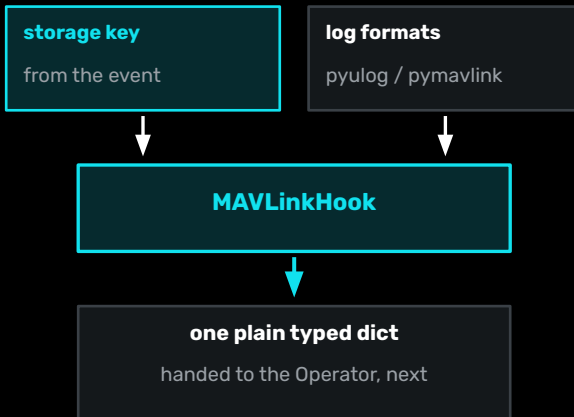
# MAVLinkHook: how to read a flight



```
class MAVLinkHook(BaseHook):
```

```
def download_log(self, bucket, key) -> Path:
    ... # the key came from the event
```

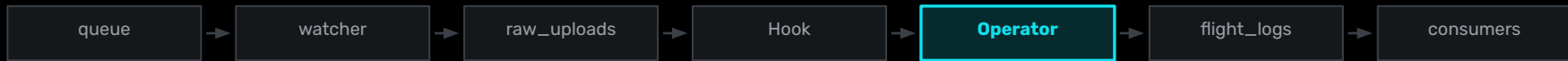
```
def parse_ulog(self, path) -> dict:
    ulog = ULog(str(path)) # .ulg via pyulog, .tlog via pymavlink
    return {
        "flight_id": ..., "airframe": ...,
        "duration_s": ..., "messages": {...},
        "errors": [...], "battery_min_v": ...,
    }
```



- Both dialects, one schema. No DAG ever imports a parser.
- Knows storage and the log format. Knows nothing about schedules or events.

**The event carried a key. The Hook turns that key into a flight.**

# The Operator: one gate, one promise



```
class TelemetryIngestOperator(BaseOperator):
    def execute(self, context):
        local = hook.download_log(bucket, raw_key) # 1 fetch
        parsed = hook.parse_u_log(local) # 2 validate
        store.write(parsed_key, parsed, # 3 write once
                    replace=True)
        yield Metadata(flight_logs, # 4 the event
                      extra={"parsed_key": parsed_key,
                            "flight_id": parsed["flight_id"]})
```

**An Asset event is a promise that the data behind it is usable.**

- Woken by raw\_uploads. Uses the Hook. Emits flight\_logs. The only piece that knows both planes.
- Validate once, at this gate. A bad parse fails the task and emits nothing.
- The write is idempotent, so a retry can never double-write a flight.

**The Hook knows how to read a flight. The Operator decides when, and tells the platform it happened.**

# Downstream DAGs evaluates Asset on arrival



```
@dag(schedule=[flight_logs], catchup=False)
def detect_flight_anomalies():

    @task
    def load_flight(**ctx):
        ev = ctx["triggering_asset_events"][flight_logs]
        return read(ev[-1].extra["parsed_key"])

    @task
    def detect(flight):
        # power . position . vibration . radio link
        return anomalies
```

- No cron. No ExternalTaskSensor. The schedule IS the Asset.
- Runs exactly once per flight, exactly when it exists.
- The scalar rules are a preflight checklist in reverse: power, position, vibration, radio link.
- Heavier tiers are separate consumers on the same Asset.

**No cron. No ExternalTaskSensor. The schedule IS the Asset.**



# The waiting problem



**50 ms**

of actual work, every 30 seconds



**30+ min**

of a worker slot held, on the tail



## Upload Latency Window

Between wheels-down and upload-complete: seconds to half an hour, depending on signal at the landing site.



## Worker Slot Lock

A classic sensor occupies its slot for the entire window.



## The Deferrable Goal

The goal: the check without the occupation.



# What waiting costs



## One worker slot, one slow upload: held for 30 minutes



cyan slivers: the 50 ms checks. All of them together add up to under three seconds of work.

everything else: a slot that is held, asleep, unavailable.



one interval, magnified:  
50 ms of work, then 29.95 s of nothing

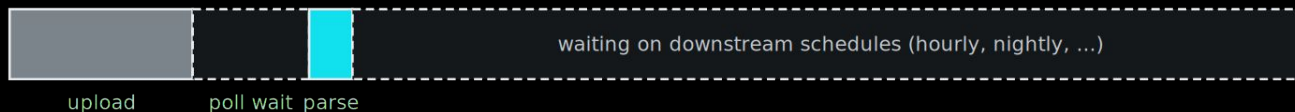
**Now multiply by every aircraft still uploading.**

# Where the hours went



## POLLING

45 min-hours to insight



## ASSETS

≈ 2 min after upload



■ work        waiting on a clock    ■ upload (physics: unchanged)

*illustrative; segment widths not to scale*

# Polling vs Assets, side by side



## POLLING

## ASSETS

### What starts work

The clock

The flight

### First sight of a flight

Up to a full poll interval later

Seconds after the marker lands

### Half-parsed records

A structural hazard

Impossible by construction

### Worker slots while waiting

One per waiting flight

Near zero; the Triggerer holds the waits

### Cross-DAG dependencies

ExternalTaskSensor to a DAG id + timing

Asset producer to consumer

### Adding an evaluation

Edit the producer, coordinate schedules

One new DAG, zero edits elsewhere

### Reprocessing history

Re-run schedules by hand

Re-emit the facts; the graph re-runs itself

# Key Takeaways



## 01

### Event-Driven Execution

Let the flight start the work, not the clock. Session-shaped, edge-originated data has no schedule of its own. In Airflow 3 the completed session becomes an Asset event, and the pipeline wakes the moment it exists.

## 02

### Zero Coupling Assets

One Asset, many events, zero coupling. Validate once at the producer gate and emit the event only when the data is usable, so every consumer trusts it. Adding an evaluation is one new DAG with no edits anywhere else.

## 03

### Language of Events

Airflow 3 speaks the language of events. Composable schedules, event metadata, and watchers that let the edge push make polling a choice, not a default. The last clock left in the pipeline is a safety net, not a mechanism.

# Questions?

**Charles Adetiloye**

MavenCode

**Bryan Johns**

MavenCode

[mavencode.com](https://mavencode.com)